

NON VALIDE

RAPPORT D'AUDIT QA

Audit End-to-End Application **ARPT** Guinee

Analyse complete de la qualite, de la securite et de la
conformite de l'application ARPT - Autorite de
Regulation des Postes et Telecommunications de
Guinee

Organisation :	ARPT Guinee
Date audit :	19 mai 2026
Version app :	0.2.0
Stack :	Next.js 16 + Express 4 + PostgreSQL 16
Verdict :	NON VALIDEE - Corrections critiques requises

Table des matieres

1. Resume executif	3
2. Portee et methodologie	4
2.1 Perimetre de l'audit	4
2.2 Critères de validation	4
3. Audit securite - Vulnerabilites critiques	4
3.1 Prise de controle via l'endpoint d'inscription	5
3.2 Fuite de mots de passe dans l'API utilisateurs	5
3.3 Secret JWT code en dur dans le source	5
3.4 Assignation en masse (Mass Assignment)	5
3.5 RBAC non applique sur les routes backend	6
4. Audit frontend - Interface utilisateur	7
4.1 Erreurs d'execution (crash)	7
4.2 Duplication massive de code	7
4.3 Donnees reglementaires fabriquees	7
4.4 Problemes d'ergonomie et d'accessibilite	7
5. Audit backend - API et logique metier	8
5.1 Endpoints non fonctionnels	9
5.2 Permissions RBAC incorrectes	9
5.3 Problemes de performance	9
6. Audit base de donnees	10

6.1 Absence de migrations Prisma	10
6.2 Regles onDelete absentes	10
6.3 Index manquants	11
7. Audit infrastructure Docker	11
7.1 Secrets en clair	11
7.2 Conteneurs non durcis	11
7.3 Port PostgreSQL expose	12
8. Plan de correction prioritaire	12
8.1 Phase 1 - Corrections critiques (20h, bloquant)	12
8.2 Phase 2 - Corrections haute priorite (30h)	13
8.3 Phase 3 - Ameliorations moyennes (20h)	13
8.4 Phase 4 - Ameliorations continues (10h)	14
9. Verdict final	14

1. Resume executif

Le present rapport synthetise les resultats de l'audit qualite end-to-end realise sur l'application ARPT Guinee (Autorite de Regulation des Postes et Telecommunications). L'audit a couvert l'ensemble de la chaine applicative : interface utilisateur (27 vues), API backend (30 routes), base de donnees (33 modeles Prisma), systeme d'authentification double (NextAuth + JWT), et l'infrastructure Docker. L'objectif est de determiner si l'application est prete pour un deploiement en production.

L'audit a revele des deficiences majeures sur les plans de la securite, de la fiabilite fonctionnelle et de la qualite du code. Des vulnerabilites critiques permettent la prise de controle du systeme par un attaquant non authentifie, des donnees reglementaires sont fabriquees en dur dans le code, et plusieurs fonctionnalites cles sont inoperantes. L'application ne peut etre validee pour un deploiement en production en l'etat actuel.

Domaine	CRITIQUE	HAUT	MOYEN	BAS	Total
Frontend (27 vues)	4	12	18	15	49
Backend (30 routes)	6	10	12	8	36
Securite / Auth	5	9	10	7	31
Docker / Infra	5	5	4	4	18
TOTAL	20	36	44	34	134

Tableau 1 : Synthese des anomalies par domaine et severite

VERDICT : APPLICATION NON VALIDE POUR LA PRODUCTION

L'application presente 20 anomalies de severite critique qui compromettent la securite, l'integrite des donnees reglementaires et la fiabilite operationnelle. Un plan de correction prioritaire est necessaire avant toute mise en production. L'estimation de l'effort de correction est d'environ 80 heures de developpement pour les anomalies critiques et de haute severite.

2. Portee et methodologie

2.1 Perimetre de l'audit

L'audit a ete conduit selon une approche systematique couvrant l'ensemble des composants de la solution technique. Chaque couche a ete analysee individuellement puis en interaction avec les autres couches, afin d'identifier les defauts isoles ainsi que les problemes d'integration. L'analyse porte sur le code source tel qu'il existe dans le depot GitHub au moment de l'audit, ainsi que sur les fichiers de configuration Docker et les variables d'environnement.

Composant	Technologie	Fichiers analyses	Methode
Frontend	Next.js 16.1.1 + React 19	27 vues + 40 composants UI	Revue de code statique
Backend API	Express 4.21 + Prisma 6.19	30 routes + 4 middlewares	Revue de code + analyse securite
Base de donnees	PostgreSQL 16 + Prisma ORM	2 schemas (33 modeles)	Analyse schema + index
Authentification	NextAuth 4 + JWT/jose	5 fichiers auth	Test d'intrusion logique
Infrastructure	Docker Compose + Caddy	3 Dockerfiles + compose	Audit configuration

Tableau 2 : Perimetre detaille de l'audit

2.2 Critères de validation

Chaque anomalie est classee selon quatre niveaux de severite, en fonction de son impact sur la securite, la fonctionnalite ou la qualite percue de l'application. Les anomalies CRITIQUES sont celles qui entrainent un crash, une faille de securite exploitable ou une corruption de donnees. Les anomalies HAUTES affectent des fonctionnalites cles ou creent des risques de securite importants. Les anomalies MOYENNES dégradent l'experience utilisateur sans bloquer les workflows. Les anomalies BAS concernent la qualite du code et la maintenabilite.

3. Audit securite - Vulnerabilites critiques

L'audit securite a revele des vulnerabilites graves qui permettent a un attaquant non authentifie de prendre le controle total de la plateforme. Ces failles sont exploitables sans competences techniques avancees et compromettent l'ensemble des donnees de l'ARPT, y compris les informations reglementaires sensibles et les donnees personnelles des utilisateurs.

3.1 Prise de controle via l'endpoint d'inscription

La vulnerabilite la plus critique reside dans l'endpoint `POST /api/auth/register` du backend Express. Le middleware d'authentification exclut les chemins commençant par `/api/auth/`, ce qui signifie que cet endpoint est accessible sans aucune authentification. Or, la route accepte le champ `role` dans le corps de la requete sans restriction, permettant a n'importe quel attaquant de creer un compte avec le role `dg` (Directeur General), qui detient les privileges d'administration maximums sur la plateforme.

En exploitant cette faille, un attaquant peut creer un compte DG, acceder a l'ensemble des donnees du systeme (y compris les mots de passe hashes des autres utilisateurs, voir section 3.2), modifier les seuils reglementaires de QoS, supprimer des operateurs et leurs historiques, et generer des rapports frauduleux. Cette vulnerabilite constitue un risque juridique majeur pour l'ARPT dans le cadre de sa mission reglementaire.

3.2 Fuite de mots de passe dans l'API utilisateurs

L'endpoint `GET /api/users` retourne les hashes de mots de passe lorsque le requérant a le role DG. Dans le fichier `users.ts` (ligne 124), la liste des utilisateurs est retournee directement sans filtrage du champ `motDePasse` pour le role DG, alors que ce filtrage est correctement effectue pour le role agent. Combine avec la vulnerabilite d'inscription non controlee, un attaquant peut obtenir les hashes de tous les mots de passe du systeme, facilitant des attaques par force brute ou par dictionnaire.

3.3 Secret JWT code en dur dans le source

Le secret JWT utilise pour signer les tokens d'authentification est code en dur avec une valeur par default dans le fichier `backend/src/auth/jwt.ts` (ligne 8) : `arpt-guinee-jwt-secret-2025`. Si la variable d'environnement `JWT_SECRET` n'est pas definie, cette valeur par default est utilisee, permettant a quiconque lit le code source (public sur GitHub) de forger des tokens valides pour n'importe quel utilisateur, y compris le DG. Le meme probleme existe pour NextAuth dans `src/lib/auth.ts` et dans les Dockerfiles ou les secrets sont incrustes dans les couches de l'image.

3.4 Assignation en masse (Mass Assignment)

Au moins 14 endpoints PUT/POST sur les 30 routes du backend transmettent directement le corps de la requete (`req.body`) aux operations Prisma sans validation par schema Zod. Bien que des schemas de validation existent dans le fichier `utils/validations.ts`, ils ne sont pas appliques sur les endpoints PUT, et parfois meme ignores sur les endpoints POST (par exemple, `decisions.ts` importe `decisionCreateSchema` mais ne l'utilise pas). Un attaquant peut ainsi injecter des champs arbitraires comme `role`, `operateurId`, ou `creeParId` pour escalader ses privileges ou manipuler les

donnees.

3.5 RBAC non applique sur les routes backend

Le systeme de controle d'accès basé sur les rôles (RBAC) est défini dans `backend/src/middleware/rbac.ts` mais n'est appliqué à aucune des 30 routes du backend. Toutes les routes utilisent uniquement `requireAuth()` qui vérifie seulement l'authentification, sans vérifier les autorisations. Tout utilisateur authentifié, y compris un compte public, peut accéder à n'importe quel endpoint, y compris la suppression d'opérateurs, la modification des seuils réglementaires et la génération de sanctions.

ID	Vulnerabilite	Fichier	Impact
SEC-01	Inscription non controlee avec choix du role	auth/routes.ts:197	Prise de controle totale
SEC-02	Secret JWT code en dur	auth/jwt.ts:8	Forge de tokens arbitraires
SEC-03	Fuite de hashes de mots de passe	users.ts:124	Compromission comptes
SEC-04	Mass Assignment sur 14 endpoints	Multiple fichiers	Escalade de privileges
SEC-05	RBAC jamais applique	index.ts	Acces non autorise total
SEC-06	Secrets en clair dans Docker Compose	docker-compose.yml	Acces DB direct
SEC-07	prisma db push --accept-data-loss en production	docker-entrypoint.sh	Perte de donnees
SEC-08	JWT retourne dans le corps de la reponse	auth/routes.ts:72	Vol de token via XSS
SEC-09	Cookies non securises en HTTP	auth/routes.ts:53	Interception session
SEC-10	Pas de limitation de taux sur reclamations publiques	public.ts:127	Spam / DoS

Tableau 3 : Inventaire des vulnerabilites de securite critiques et hautes

4. Audit frontend - Interface utilisateur

L'audit du frontend a porte sur les 27 composants de vue, les composants UI reutilisables et le systeme de navigation. L'interface est construite avec Next.js 16, React 19, Tailwind CSS 4 et la bibliotheque shadcn/ui. L'analyse a revele des problemes de fiabilite fonctionnelle, de qualite du code et d'experience utilisateur qui affectent significativement la valeur percue de l'application.

4.1 Erreurs d'execution (crash)

Quatre anomalies provoqueront des crashes a l'execution. La plus evidente est l'appel a `loadAuditLogs()` dans le bouton Actualiser de la vue audit-logs, alors que cette fonction n'est ni importee ni definie dans le composant. Un `ReferenceError` sera leve a chaque clic. La vue analyse utilise `Math.random()` dans le chemin de rendu, provoquant des valeurs differentes a chaque re-rendu et des graphiques qui clignotent. Les vues dashboard et rapports contiennent des donnees hardcoded qui affichent des informations reglementaires fabriquees au lieu de donnees reelles.

4.2 Duplication massive de code

Les composants utilitaires `StatusBadge`, `AccessDenied`, `KpiCard` et `LoadingSkeleton` sont copies-collés dans les 27 fichiers de vue, generant environ 1200 lignes de duplication pure. Des incoherences existent deja entre les differentes copies (espacement different dans `KpiCard` selon la vue). De plus, 20 des 27 vues importent environ 40 icones lucide et tous les composants recharts alors qu'elles n'en utilisent que 5 a 8, gonflant inutilement le bundle et ralentissant la compilation.

4.3 Donnees reglementaires fabriquees

Plusieurs indicateurs clés affichés sur le tableau de bord du DG sont hardcoded dans le code plutot que calcules a partir des donnees reelles. Le delai moyen de traitement SLA est figé a 4.2 jours (fichier `dashboard-view.tsx`, ligne 564). La synthese executive DG contient des affirmations statiques comme "MTN Guinee sous seuil reglementaire (93,4% vs 95%)" qui ne refletent pas la realite des mesures. Les rapports precedents sont des lignes JSX hardcoded dans `rapports-view.tsx`. Ces informations fabriquees constituent un risque de conformite reglementaire majeur pour l'ARPT, qui est l'autorite chargee de verifier la QoS des operateurs.

4.4 Problemes d'ergonomie et d'accessibilite

L'audit a identifie 12 vues sans etat de chargement (l'utilisateur voit un ecran blanc jusqu'a l'arrivee des donnees), des recherches sans debounce (chaque frappe declenche un appel API), des grilles non responsives qui debordent sur

ecran etroit, et aucune etiquette ARIA sur les boutons iconiques (plus de 40 instances). Le systeme d'internationalisation est configure mais aucune vue ne l'utilise : tout le texte est en francais hardcoded dans le JSX, rendant l'application unilingue malgre la pretention i18n. Par ailleurs, le composant guichet-view depasse 900 lignes et melange logique de liste, creation de dossier, upload de fichiers, configuration SLA et escalade, rendant la maintenance quasi impossible.

ID	Anomalie	Severite	Vue(s) concerne(s)
FE-01	loadAuditLogs() non definie - crash au clic	CRITIQUE	audit-logs-view
FE-02	Math.random() dans le rendu - graphiques instables	CRITIQUE	analyse-view
FE-03	Delai SLA hardcoded 4.2 jours	CRITIQUE	dashboard-view
FE-04	Rapports precedents hardcoded en JSX	CRITIQUE	rapports-view
FE-05	1200 lignes de code duplique (4 composants x 27 vues)	HAUT	Toutes les vues
FE-06	Import massif non utilise (40 icones, recharts)	HAUT	20 vues
FE-07	Secret cron dans le code client	HAUT	dashboard, guichet
FE-08	12 vues sans etat de chargement	MOYEN	12 vues
FE-09	Recherche sans debounce	MOYEN	users, operateurs...
FE-10	i18n configure mais jamais utilise	MOYEN	Toutes les vues
FE-11	XSS dans l'aperçu impression decisions	HAUT	decisions-view
FE-12	Pas de pagination dans 6 tableaux	MOYEN	6 vues

Tableau 4 : Inventaire des anomalies frontend principales

5. Audit backend - API et logique metier

L'audit du backend a porte sur les 30 fichiers de routes Express, les 4 middlewares et les 4 utilitaires. L'API est construite avec Express 4.21 et Prisma 6.19 pour l'accès à la base de données PostgreSQL. L'audit a revele des problemes de securite majeurs (deja detailles en section 3), des deficiences fonctionnelles sur plusieurs endpoints cles, et des problemes de performance qui

affecteront la scalabilite de l'application.

5.1 Endpoints non fonctionnels

L'endpoint de restauration de version (POST /api/versions/restore) est un no-op : il enregistre une entree d'audit et retourne les donnees de la version, mais ne met jamais a jour l'entite cible. La restauration ne fait donc rien. L'endpoint de suppression d'operateur execute 10 suppressions sequentielles sans transaction : si une operation echoue en cours de route, les donnees sont laissees dans un etat incoherent. De plus, cette suppression omet les tables Sanction et RapportGenere, causant une erreur de contrainte de cle etrangere. La numerotation sequentielle des references (dossiers, decisions, sanctions) utilise un compteur base sur la longueur du tableau cote client, creant une condition de concurrence : deux requetes simultanees peuvent generer la meme reference.

5.2 Permissions RBAC incorrectes

Meme si le RBAC n'est pas applique globalement (voir section 3.5), les middleware authorize() utilises sur certaines routes contiennent des erreurs. La suppression d'operateur utilise la permission 'update' au lieu de 'delete'. La suppression de templates utilise 'update'. Les routes de rapports generes utilisent le nom de ressource 'templates' au lieu de 'rapports-generes'. La creation de seuils QoS utilise 'update' au lieu de 'create'. La route antennes utilise uniquement requireAuth() sans aucune autorisation, permettant au role public d'acceder aux donnees d'antennes sensibles.

5.3 Problemes de performance

Six endpoints effectuent des requetes double : une requete paginee pour les donnees, puis une requete non paginee pour calculer des statistiques (zones-blanches, reclamations, tarifs, sanctions, fai, couverture). Avec des milliers d'enregistrements, ces requetes non bornees chargeront l'integralite des donnees en memoire uniquement pour compter ou sommer des valeurs. L'approche correcte serait d'utiliser les fonctions Prisma aggregate(), groupBy() ou count(). L'endpoint dashboard effectue 5 requetes findMany sans limitation. L'endpoint export charge l'integralite des donnees pour chaque type d'export. Ces problemes provoqueront une degradation progressive des performances a mesure que le volume de donnees augmente.

ID	Anomalie	Severite	Fichier(s)
BE-01	Version restore est un no-op	HAUT	versions.ts:64
BE-02	DELETE operateur sans transaction	CRITIQUE	operateurs.ts:183

BE-03	Permissions RBAC incorrectes (5 routes)	HAUT	Multiple fichiers
BE-04	Condition de concurrence sur references	HAUT	dossiers, decisions, sanctions
BE-05	6 endpoints avec requetes non bornees	MOYEN	6 routes
BE-06	Schemas Zod existent mais non utilises (PUT)	HAUT	10+ routes
BE-07	Dashboard avec 5 findMany sans pagination	MOYEN	dashboard.ts
BE-08	Notifications creees une par une en boucle	MOYEN	4 routes
BE-09	Reponse inconsistante (3 formats differents)	MOYEN	Multiple routes
BE-10	Donnees benchmark exportees hardcoded	MOYEN	export.ts:127

Tableau 5 : Inventaire des anomalies backend principales

6. Audit base de donnees

L'audit de la base de donnees a porte sur les deux schemas Prisma (frontend et backend), la strategie de migration, les index et les regles d'integrite referentielle. Les deux schemas sont heureusement identiques, evitant tout probleme de derive. Cependant, des lacunes importantes ont ete identifiees en matiere d'integrite des donnees et de performance des requetes.

6.1 Absence de migrations Prisma

Le projet utilise exclusivement `prisma db push` au lieu de `prisma migrate`. L'option `--accept-data-loss` est meme passee dans les scripts d'entree Docker, ce qui signifie qu'un changement de schema en production peut silencieusement supprimer des colonnes ou des tables entieres sans avertissement. Il n'y a aucun historique de migrations, aucune possibilite de rollback, et aucune tracabilite des evolutions du schema. Cette approche est explicitement deconseillee par Prisma pour les environnements de production.

6.2 Regles onDelete absentes

La plupart des relations du schema Prisma ne specifient pas de regle `onDelete`. Cela signifie que la suppression d'un utilisateur echouera si celui-ci a des enregistrements `AuditLog`, `Notification`, `Dossier` ou `Reclamation` associes, car PostgreSQL refusera la violation de cle etrangere. Les regles appropriees

devraient être `onDelete: SetNull` pour les relations d'audit (préservé l'historique) et `onDelete: Cascade` pour les notifications. L'absence de ces règles obligera à des suppressions manuelles en cascade ou à des contournements dangereux en production.

6.3 Index manquants

Plusieurs champs fréquemment filtrés dans les requêtes ne disposent pas d'index en base de données, ce qui entraînera des scans séquentiels complets à mesure que le volume de données augmente. Les champs critiques sans index incluent : `User.role` (filtrage du tableau de bord), `Declaration.operateurId + periode` (recherche de déclarations), `QoSMeasure.operateurId` (requêtes QoS), `Reclamation.status` (filtrage des réclamations ouvertes), et `AuditLog.userId + createdAt` (requêtes d'audit). L'ajout de ces index est essentiel pour maintenir des temps de réponse acceptables en production.

7. Audit infrastructure Docker

L'audit de l'infrastructure a porté sur les 3 Dockerfiles, le fichier `docker-compose.yml`, le Caddyfile et les variables d'environnement. L'application est déployée en architecture 3 tiers : frontend Next.js (port 3002), backend Express (port 4000), et PostgreSQL 16 (port 5433). L'analyse a révélé des problèmes de sécurité dans la configuration des conteneurs et des pratiques de construction non conformes aux standards de production.

7.1 Secrets en clair

Tous les secrets (mot de passe PostgreSQL, `JWT_SECRET`, `NEXTAUTH_SECRET`) sont codés en dur dans le fichier `docker-compose.yml`, les Dockerfiles et les variables d'environnement. Le Dockerfile frontend incorpore les secrets dans les couches de l'image (lignes 22-26), où ils peuvent être extraits via `docker history`. Il n'y a aucune utilisation de Docker secrets, de vault, ou même de fichier `.env` avec permissions restrictives. Le mot de passe par défaut `arpt2025` est utilisé pour tous les comptes sans politique de complexité.

7.2 Conteneurs non durcis

Le backend Dockerfile installe `tsx` et `prisma CLI` dans l'image de production (ligne 59), élargissant inutilement la surface d'attaque. Il n'y a pas de fichier `.dockerignore` pour le backend, ce qui signifie que tout le contexte (`node_modules`, `.env`, `.git`) est envoyé au daemon Docker. Le conteneur frontend n'a pas de healthcheck, contrairement au backend et à PostgreSQL. Le Caddyfile écoute sur le port 81 sans TLS, et le CSP autorise `unsafe-inline` et `unsafe-eval`, affaiblissant significativement la protection contre les attaques XSS. Les cookies de session ne sont pas marqués `secure`, permettant leur interception sur un réseau non chiffré.

7.3 Port PostgreSQL expose

Le port PostgreSQL (5433) est expose sur l'hote avec un mot de passe faible cod en dur. Cela permet un acces direct a la base de donnees depuis le reseau hote sans aucune authentication supplementaire. En production, la base de donnees ne devrait etre accessible que depuis le reseau Docker interne, sans mappage de port vers l'hote.

8. Plan de correction prioritaire

Le plan de correction est organise en quatre phases, de la plus critique a la moins urgente. Les corrections de phase 1 sont des prerequis absolus avant toute mise en production. Les phases 2 et 3 sont necessaires pour une mise en production acceptable. La phase 4 concerne l'amelioration continue et peut etre planifiee apres le lancement.

8.1 Phase 1 - Corrections critiques (20h, bloquant)

Priorite	Correction	Effort	Impact
P1-01	Restreindre l'inscription : seul le DG/agent peut creer des comptes, role public uniquement pour l'auto-inscription	3h	Elimine la prise de controle
P1-02	Supprimer les fallbacks de secrets JWT/NextAuth, ajouter une validation au demarrage	2h	Elimine la forge de tokens
P1-03	Appliquer les schemas Zod sur TOUS les endpoints PUT/POST	4h	Elimine le mass assignment
P1-04	Appliquer le middleware authorize() sur toutes les routes backend	3h	Active le controle d'accès
P1-05	Filtrer les mots de passe dans GET /api/users pour le role DG	0.5h	Elimine la fuite de hashes
P1-06	Wrapper DELETE operateur dans une transaction Prisma	1h	Elimine l'incoherence de donnees
P1-07	Retirer le token du corps de la reponse login	0.5h	Reduit l'exposition du token
P1-08	Ajouter une limitation de taux sur POST /api/public/reclamations	1h	Protege contre le spam
P1-09	Retirer --accept-data-loss, creer des migrations Prisma	3h	Protege les donnees en production

P1-10	Corriger loadAuditLogs() et Math.random() dans le frontend	2h	Elimine les crashes
-------	--	----	---------------------

Tableau 6 : Phase 1 - Corrections critiques bloquantes

8.2 Phase 2 - Corrections haute priorite (30h)

Priorite	Correction	Effort	Impact
P2-01	Extraire les composants partages (StatusBadge, KpiCard, etc.) dans un module commun	3h	Maintenabilite
P2-02	Nettoyer les imports inutilises dans les 27 vues	4h	Performance compilation
P2-03	Deplacer le declenchement SLA du client vers une action serveur	1h	Securite
P2-04	Ajouter la gestion d'erreurs try/catch sur tous les fetch manquants	2h	Fiabilite
P2-05	Ajouter des etats de chargement aux 12 vues qui en manquent	4h	Experience utilisateur
P2-06	Implementer la restauration de version (endpoint no-op)	2h	Fonctionnalite
P2-07	Corriger les permissions RBAC sur 5 routes (delete vs update)	2h	Controle d'accès
P2-08	Ajouter la verification de propriete sur notification mark-read	1h	Securite
P2-09	Ajouter pagination aux 6 endpoints sans limitation	4h	Performance
P2-10	Remplacer les donnees hardcoded par des appels API dans le dashboard	4h	Conformite reglementaire
P2-11	Corriger le XSS dans l'aperçu impression decisions	0.5h	Securite
P2-12	Ajouter .dockerignore pour le backend, healthcheck pour le frontend	1h	Operations
P2-13	Ajouter les index manquants dans le schema Prisma	1.5h	Performance DB

Tableau 7 : Phase 2 - Corrections haute priorite

8.3 Phase 3 - Ameliorations moyennes (20h)

La phase 3 comprend les améliorations qui ne sont pas bloquantes pour la mise en production mais qui sont nécessaires pour une qualité professionnelle acceptable. Cela inclut : la définition d'interfaces TypeScript pour les objets métier (Opérateur, Dossier, Decision, etc.) en remplacement du type any omniprésent, le découpage des composants trop volumineux (guichet-view 900 lignes, dashboard-view 700 lignes), l'ajout d'aria-labels sur les boutons iconiques, l'unification du format de réponse API, l'ajout de refresh tokens avec revocation, la configuration HTTPS via Caddy, et l'amélioration de la politique de mots de passe (complexité minimum). L'estimation totale pour cette phase est d'environ 20 heures.

8.4 Phase 4 - Améliorations continues (10h)

La phase 4 regroupe les améliorations de qualité de code et d'expérience développeur qui peuvent être planifiées après le lancement. Cela inclut : le code-splitting par vue avec React.lazy, l'activation du système i18n ou sa suppression, l'ajout de debounces sur les champs de recherche, l'ajout de tests unitaires pour les composants de vue, la mise en place de logging structure (winston/pino) en remplacement des console.error, et la mise à jour de la politique CSP pour supprimer unsafe-inline et unsafe-eval. L'estimation totale pour cette phase est d'environ 10 heures.

9. Verdict final

À l'issue de cet audit end-to-end, l'application ARPT Guinée ne peut pas être validée pour un déploiement en production. Les 20 anomalies critiques identifiées représentent des risques sécuritaires exploitables, des déficiences fonctionnelles majeures et des non-conformités réglementaires qui compromettent la mission de l'ARPT en tant qu'autorité de régulation.

VERDICT : NON VALIDE POUR LA PRODUCTION

134 anomalies identifiées (20 critiques, 36 hautes, 44 moyennes, 34 basses). Effort de correction estimé : 80 heures (Phases 1-3).

L'application peut être validée après complétion de la Phase 1 (corrections critiques) et validation par un audit de sécurité de pénétration.

Les conditions minimales pour une validation sont : (1) complétion de toutes les corrections de Phase 1, (2) test de pénétration réussi sur les vecteurs d'attaque

identifies, (3) verification que les donnees reglementaires affichees proviennent exclusivement de la base de donnees et non de valeurs hardcoded, et (4) validation fonctionnelle end-to-end de tous les workflows critiques (authentification, creation de dossier, mesure QoS, sanction). L'ARPT est invitee a mettre en place un processus de revue de code systematique et une integration continue avec tests automatises avant toute nouvelle livraison.